



AVALIAÇÃO TÉCNICA

O que este sistema realmente é.

Uma avaliação do CRC feita lendo o código e medindo — não lendo a documentação dele. Com o que está sólido, o que é frágil, e um erro que quase entrou neste documento por eu ter confiado num comando de busca.

O inventário é automático: contar módulos, medir linhas, cruzar cada arquivo com o teste correspondente, procurar leitura de banco sem filtro de organização.

O alarme falso que não entrou aqui

Na verificação de isolamento entre clínicas, o script acusou **72 leituras sem filtro de tenant** — um número que, escrito sem conferência, seria um alarme grave e falso. Fui olhar caso a caso: todas usavam filtro montado antes (`const filtros = ...`) ou um helper (`daOrganizacao()`), que o padrão de busca não enxerga. **Nenhuma das 72 era real.**

Isso define a regra do resto do documento: onde aparece um número, ele passou por inspeção. Onde não deu para inspecionar, está escrito que não deu.

O tamanho do sistema

CAMADA	QUANTIDADE
Domínio — regras puras	49 módulos · 23.296 linhas
Serviços	54 módulos
Plataforma de IA	11 módulos · ~3.650 linhas
Telas	44 componentes · 28 abas
Banco	91 tabelas · 40 migrations
Testes	107 arquivos · 1.754 testes
E2E em navegador real	7 suítes

Para calibrar: são mais linhas de **regra de negócio pura** — sem banco, sem rede, sem relógio — do que muitos produtos comerciais têm de código total.

As camadas são reais

As 23.296 linhas de domínio não tocam banco, rede nem relógio. A regra de encaixe, de risco de falta e de aceitação são testáveis sem subir nada.

É o que explica 1.754 testes rodarem em 30 segundos. Sistema com regra embolada no acesso a dados não consegue isso — e o time para de rodar teste.

Idempotência é constraint

Não duplicar mensagem, oportunidade e sincronização está em índice único no Postgres. Código com bug não consegue duplicar: recebe erro.

Inclui índice parcial, que permite uma oportunidade fechada reabrir no mês seguinte sem quebrar a chave.

O banco-fake confere o schema

Os testes rodam contra um Postgres de mentira que valida nome de coluna contra os arquivos SQL — na escrita e na leitura.

Pega o que TypeScript e lint não alcançam. Encontrou três colunas inexistentes nas primeiras horas, uma num arquivo já commitado.

Os comentários explicam o porquê

O código registra a decisão, e não a mecânica — o tipo de frase que impede alguém de “simplificar” uma regra seis meses depois sem saber o que ela protegia.

O ponto mais forte: o isolamento está no banco, não na disciplina

Contra um Postgres de verdade, os testes provam que o banco **recusa fisicamente** paciente de uma organização numa clínica de outra, conversa cruzada, mensagem numa conversa alheia e job do agente apontando para fora do tenant. São chaves estrangeiras compostas, e não where bem escrito — a diferença entre “nós tomamos cuidado” e “não é possível”. Só a segunda sobrevive a um programador com pressa.

Os E2E cobrem o que dói. Não são teste de vitrine — as sete suítes rodam num navegador de verdade e verificam exatamente os pontos onde uma falha seria cara:

Kill switch	Liga e desliga os envios, de verdade
Multi-clínica	Usuário da unidade A NÃO enxerga paciente da B — e o admin enxerga as duas
Sessão	Sobrevive a reload; senha errada não revela se o e-mail existe
Radar	Mostra o esperado SEPARADO do potencial
Base vazia	A tela ensina o próximo passo, em vez de mostr

A plataforma de IA tem peça que produto comercial costuma não ter

MÓDULO	O QUE FAZ	LINHAS
executor	Roda as ferramentas que a IA pediu	893
turno	Um turno de conversa, do contexto à decisão	747
supervisor	Avalia a decisão antes de ela sair	367
replay	Reprocessa uma decisão antiga com o código de hoje	367
tracing	O rastro de cada chamada	362
contexto	O que a IA vê — e o que ela não vê	327
laço	Decidir → executar ferramenta → decidir de novo	294

replay é o incomum: permite responder “por que ela fez isso em março?” rodando de novo a mesma entrada com o código de hoje. Quase ninguém constrói isso antes de precisar.

A maior lacuna do sistema: cobranças sem teste

862 linhas · 7 escritas no banco · importação de CSV · ZERO teste. Nem direto, nem indireto — o nome não aparece em nenhum dos 107 arquivos de teste.

É o único módulo que mexe com dinheiro de paciente sem nenhuma rede. Importa planilha, classifica status, gera prévia, grava cobrança e roda varredura. Um erro de leitura aqui não quebra: ele cobra a pessoa errada, ou o valor errado.

Se fosse para consertar uma coisa só neste sistema, seria esta.

Cinco domínios com regra de verdade e sem teste próprio

MÓDULO	LINHAS	FUNÇÕES
regras.ts	390	9
validar.ts	232	13
churn.ts	233	1
proxima-acao.ts	210	2
melhor-horario.ts	207	2

Os outros doze módulos “sem teste” são tipo, rótulo e formatação — não preocupam. Estes cinco decidem coisa.

E os dois maiores serviços não têm suíte dedicada: mensagens.ts (1.114 linhas) e sincronizacao.ts (943). São cobertos de lado pelos testes de integração e pelos E2E, o que atenua — mas são os dois maiores arquivos de serviço do sistema.

1

O sistema não roda

Produção está **28 commits atrás**, com **0 pacientes**, e o CI **nunca rodou** nesta sequência. Este risco domina todos os outros — e não é técnico. Um sistema com 1.754 testes que nunca atendeu um paciente vale o que vale um sistema que não existe.

2

Cobrança sem teste, tocando dinheiro

A única dívida técnica que eu trataria como urgente.

3

A escrita na agenda nasce desligada

A flag `dental_office_writeback` está false por padrão, com o comentário “fica off no primeiro rollout”. Qualquer promessa de “o sistema marca sozinho” precisa ser corrigida antes de ser feita.

4

Voz e pagamento são arquitetura sem provedor

Os contratos estão completos e corretos. São inúteis até alguém contratar — o que é a decisão certa, mas precisa ser dito em voz alta.

O que eu faria, em ordem

1. Empurrar os 28 commits e observar o CI — nada aqui afirma que ele passa. 2. Escrever a suíte de cobranças: planilha torta, valor com vírgula, linha duplicada, status desconhecido. 3. Deploy e sincronização do Dental Office; enquanto a base tiver 0 pacientes, nenhum número significa nada. 4. Testes para os cinco domínios, começando por `regras.ts`. 5. Só então: provedor de voz e de pagamento, se e quando fizer sentido comercial.

Não falta código. Faltam **duas credenciais** e um push.

O que salta nesta avaliação é uma assimetria: a qualidade do que está construído é muito maior do que a maturidade operacional.

Um sistema com isolamento provado por chave estrangeira, replay de decisão de IA, idempotência em constraint e 1.754 testes — que nunca falou com um paciente.

Isso não é crítica ao trabalho. É a constatação de onde está o gargalo. A parte difícil — a que costuma matar projeto de software — está feita e verificada. O que falta é operacional, e some numa tarde.